

Personal Library — BPMN 2.0 Process Definition (Static Export)

Reference visualization of document_ingestion_rag.bpmn — not run by an embedded engine

PROCESS IDENTIFIER	EXECUTION STATUS	LICENSE & STANDARD
Process_DocumentIngestionRAG	Not executed — no embedded Camunda/Zeebe engine in this app	GNU AGPLv3 · BPMN 2.0 with BPMNDI

Executive Summary & Pipeline Architecture

This BPMN 2.0 file is a static process definition exported from the real document-ingestion logic for architecture documentation, compliance review, and optional future Camunda Modeler import. It is NOT deployed to or executed by an embedded process engine — the running Spring Boot application has no Camunda/Zeebe dependency and performs these same steps as a plain sequential/async Java service call chain inside DocumentService.

What the Diagram Documents (mirroring the real pipeline):

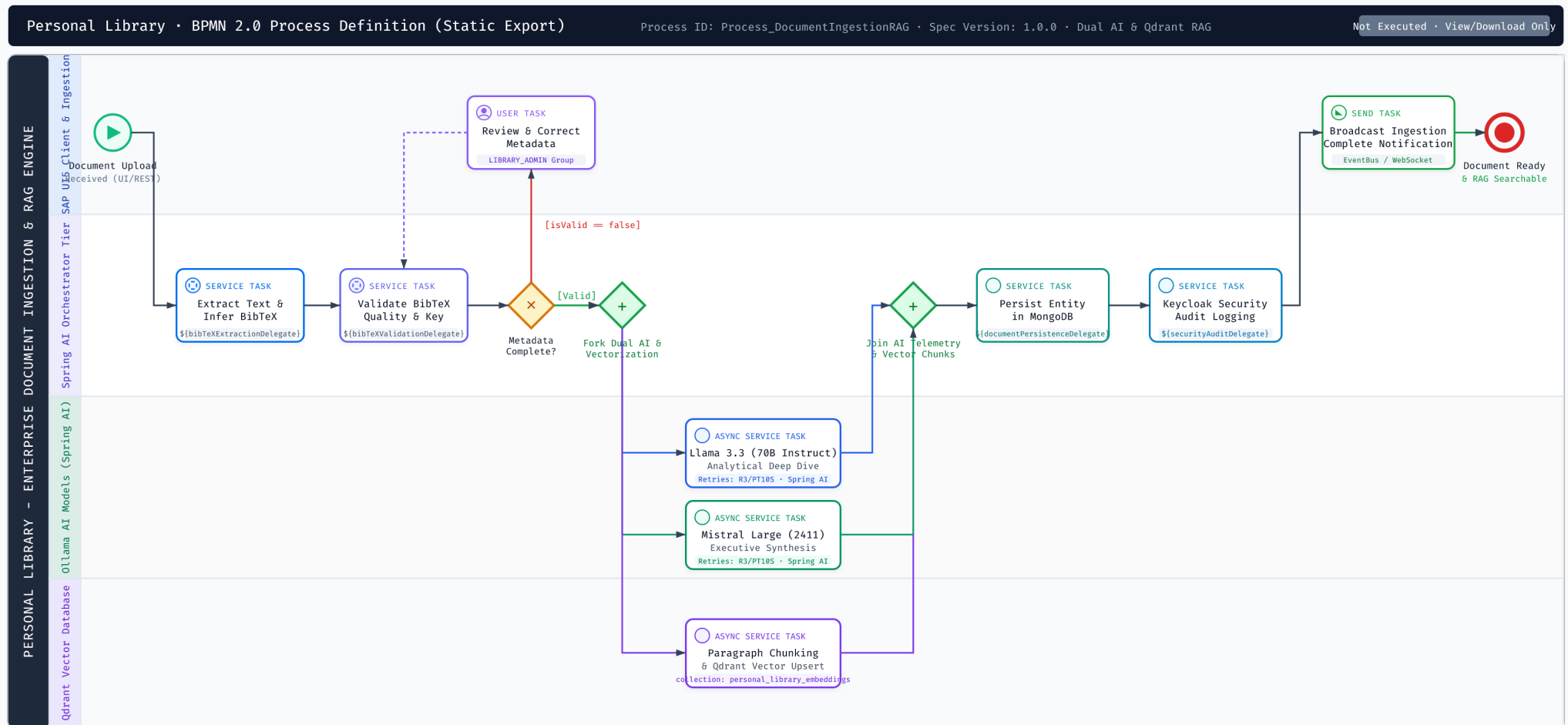
- 4 Horizontal Swimlanes: Client Ingestion (UI5/REST), Spring AI Orchestrator, Ollama Dual Models, and Qdrant Vector Store.
- Parallel Dual-Model Summarization: Ollama Llama 3.3 (70B Instruct) for an analytical deep dive and Mistral Large (2411) for an executive synthesis, run concurrently alongside paragraph-based semantic chunking for Qdrant.
- Metadata Quality Gate: An exclusive gateway branch represents the path taken when BibTeX extraction is incomplete, requiring librarian/researcher review before indexing — in the real app this is a validation check inside the same request, not a separate human task queue.
- Governance: Persistence in MongoDB plus security audit logging and real-time push notifications over the SAP UI5 EventBus.

Reference-Only Engine Compatibility (illustrative, not implemented in this repository):

- Camunda 7: Would require JavaDelegate classes bound via camunda:delegateExpression, none of which exist in this codebase.
- Camunda 8: Would require Zeebe JobWorkers, none of which exist in this codebase.

BPMN 2.0 Process Diagram (Static Export)

Visualization of the swimlanes and tasks defined in the .bpmn XML file — not a live running workflow



Process Task Catalog (From the Static .bpmn XML)

Attribute values literally present in the XML file; none are bound to running Java classes or workers

ACTIVITY / TASK NAME	XML: camunda:delegateExpression	XML: zeebe task type	SWIMLANE	EXECUTION	RETRIES
Extract Text & BibTeX	<code>\${bibTeXExtractionDelegate}</code>	extract-bibtex-task	Spring AI Orchestrator	Synchronous	3
Validate BibTeX Quality	<code>\${bibTeXValidationDelegate}</code>	validate-bibtex-task	Spring AI Orchestrator	Synchronous	None
Review & Correct Metadata	User Task (Librarian)	User Task (Human)	SAP UI5 Client	Manual Review	N/A
Fork Dual AI & Vectorization	Parallel AND Gateway	Parallel AND Gateway	Spring AI Orchestrator	Fork Split	N/A
Llama 3.3 (70B) Summary	<code>\${llamaSummarizationDelegate}</code>	llama-summary-task	Ollama AI Models	<code>asyncBefore=true</code>	R3/PT10S
Mistral Large (2411) Summary	<code>\${mistralSummarizationDelegate}</code>	mistral-summary-task	Ollama AI Models	<code>asyncBefore=true</code>	R3/PT10S
Chunk Text & Upsert Vectors	<code>\${vectorRagDelegate}</code>	qdrant-vectorization-task	Qdrant Vector DB	<code>asyncBefore=true</code>	R3/PT10S
Join AI & Vector Telemetry	Parallel AND Join	Parallel AND Join	Spring AI Orchestrator	Join Barrier	N/A
Persist Entity in MongoDB	<code>\${documentPersistenceDelegate}</code>	persist-document-task	Spring AI Orchestrator	Synchronous	3
Keycloak Security Audit Log	<code>\${securityAuditDelegate}</code>	keycloak-audit-task	Spring AI Orchestrator	Synchronous	3
Broadcast Ingestion Event	<code>\${eventBusNotificationDelegate}</code>	notify-client-task	SAP UI5 Client	Send Task	None

What These XML Attributes Mean (Reference Only)

The AI inference and Qdrant vectorization activities are annotated with `camunda:asyncBefore="true"` and `camunda:failedJobRetryTimeCycle (R3/PT10S)` in the static XML, describing how these steps WOULD behave if this definition were ever deployed to a real Camunda engine. This application does not do so: there is no Camunda Job Executor, no Zeebe broker, and no delegate/worker bean in the running Spring Boot process. In the real app, retries and async execution for these steps are handled by ordinary Java exception handling and the Spring `@Async/thread-pool` mechanisms instead.

Illustrative Only: Optional Future Camunda Integration

Neither class below exists in this repository today. Shown only as a starting point for teams who want to deploy this .bpmn file to a real Camunda engine, wired to the actual AiSummarizationService/VectorRagService APIs.

Camunda 7 (Illustrative, NOT in this repo)

Hypothetical package: com.personallibrary.workflow.delegate

```
@Component("llamaSummarizationDelegate")
public class LlamaSummarizationDelegate implements JavaDelegate {
    @Autowired
    private AiSummarizationService summarizationService;

    @Override
    public void execute(DelegateExecution execution) throws Exception {
        String title = (String) execution.getVariable("documentTitle");
        String content = (String) execution.getVariable("fullContent");
        BibTeXMetadata bibtex = (BibTeXMetadata)
            execution.getVariable("bibtex");

        SummaryRecord summary = summarizationService
            .generateSummaryForModel("llama", title, content, bibtex);

        execution.setVariable("llamaSummary", summary);
    }
}

@Component("vectorRagDelegate")
public class VectorRagDelegate implements JavaDelegate {
    @Autowired
    private VectorRagService vectorRagService;

    @Override
    public void execute(DelegateExecution execution) throws Exception {
        String docId = (String) execution.getVariable("documentGuid");
        String text = (String) execution.getVariable("fullContent");

        List<DocumentChunk> chunks =
            vectorRagService.indexDocumentChunks(docId, text);
        execution.setVariable("vectorChunks", chunks);
    }
}
```

Camunda 8 (Illustrative, NOT in this repo)

Hypothetical package: com.personallibrary.workflow.worker

```
@Component
public class DocumentIngestionZeebeWorkers {
    @Autowired
    private AiSummarizationService summarizationService;
    @Autowired
    private VectorRagService vectorRagService;

    @JobWorker(type = "llama-summary-task", autoComplete = true)
    public Map<String, Object> handleLlama(final ActivatedJob job) {
        Map<String, Object> vars = job.getVariablesAsMap();
        String title = (String) vars.get("documentTitle");
        String content = (String) vars.get("fullContent");

        SummaryRecord summary = summarizationService
            .generateSummaryForModel("llama", title, content, null);
        return Map.of("llamaSummary", summary);
    }

    @JobWorker(type = "qdrant-vectorization-task", autoComplete = true)
    public Map<String, Object> handleQdrant(final ActivatedJob job) {
        Map<String, Object> vars = job.getVariablesAsMap();
        String docId = (String) vars.get("documentGuid");
        String text = (String) vars.get("fullContent");

        int chunks = vectorRagService.indexDocumentChunks(docId,
            text).size();
        return Map.of("chunksCount", chunks, "status", "INDEXED");
    }
}
```